

Domain 5 — Data Collaboration & Protection

Complete Beginner's Master Guide | COF-C03 2026

Technical + Layman Definitions | All Subdomains | Edge Cases | Hands-On SQL |
20 Questions Per Topic | Full Answer Key

How to use this guide

1. Read ☒ Layman Explanation first — build the mental model
2. Read ☒ Technical Explanation — precise mechanics, every edge case, exam traps
3. Run every Hands-On Query block in your Snowflake free trial
4. Attempt all 20 questions per topic WITHOUT peeking at answers
5. Check the annotated Answer Key at the very end

Exam facts: Domain 5 (Data Collaboration & Protection) = 10% of the COF-C03 exam. It covers two subdomains: 5.1 Data Collaboration and Protection (Time Travel, Fail-Safe, Cloning, Replication) and 5.2 Snowflake's Data Sharing Capabilities (Secure Data Sharing, Listings, Marketplace, Reader Accounts).^{[1][2][^3]}

SUBDOMAIN 5.1 — Data Collaboration and Protection

This subdomain covers the features Snowflake provides to protect your data from accidental loss, enable historical data access, and replicate data across regions and clouds for business continuity.

TOPIC 5.1.1 — Time Travel

☒ Layman's Explanation

Imagine you have a magic "undo" button for your database. You accidentally deleted 1 million customer records at 2 PM. Time Travel lets you go back in time to 1:59 PM, see the data exactly as it was, and restore it. It's like a built-in DVR for your database — recording every change for a configurable window.

Three things you can do with Time Travel:

1. Query the past — "Show me what this table looked like 6 hours ago"
2. Clone from the past — "Give me a copy of this table as it was yesterday"

3. Restore (UNDROP) — "I deleted this table — bring it back"

☒ Technical Explanation

What it is: Time Travel is Snowflake's built-in feature that retains historical versions of modified/deleted data for a configurable retention period, allowing queries, cloning, and restoration of data at any point within that window.^[4]

Retention Periods — The Most Important Table in the Exam^{[5][6][7][4]}

Table Type	Time Travel Retention	Fail-Safe	Editions Applicable
Permanent	0 to 90 days	7 days	Enterprise & higher
Permanent	0 or 1 day (default: 1)	7 days	Standard edition
Transient	0 or 1 day (default: 1)	0 days (NO Fail-Safe)	All editions
Temporary	0 or 1 day (default: 1)	0 days (NO Fail-Safe)	All editions

Critical rules:

- The default Time Travel for ALL new tables is **1 day** — regardless of edition
- Transient and Temporary tables can NEVER have more than 1 day of Time Travel
- The Fail-Safe period is 7 days and is **NOT configurable** — Snowflake controls it
- Time Travel and Fail-Safe apply to: Tables, Schemas, and Databases
- External Tables, Internal Stages, and Temporary Tables (when session ends) do NOT retain Time Travel history after the object is dropped

The Three Time Travel Clause Options^{[8][4]}

Option	Syntax	Meaning
AT (TIMESTAMP)	AT (TIMESTAMP => 'timestamp')	Query data as it existed AT the exact timestamp
AT (OFFSET)	AT (OFFSET => -3600)	Query data N seconds in the past (negative integer = seconds ago)
AT (STATEMENT)	AT (STATEMENT => 'query_id')	Query data as it existed just BEFORE the specified query ran
BEFORE (STATEMENT)	BEFORE (STATEMENT => 'query_id')	Data state BEFORE the specified query executed

Ordering: AT means "data at that exact moment" (inclusive). BEFORE means "data before that event happened" (exclusive).

Time Travel Operations^[7]_[4]

1. Historical Query (SELECT from the past):

```
-- Query table as it was 2 hours ago
SELECT * FROM orders AT (OFFSET => -7200);

-- Query table as it was at a specific timestamp
SELECT * FROM orders AT (TIMESTAMP => '2026-01-15 14:00:00'::TIMESTAMP_TZ);

-- Query data as it was BEFORE a specific query ran
SELECT * FROM orders BEFORE (STATEMENT => '01234567-abcd-....');
```

2. UNDROP (Restore dropped objects):

```
-- Restore a dropped table (within retention window)
UNDROP TABLE orders;

-- Restore a dropped schema (restores all tables inside)
UNDROP SCHEMA sales_schema;

-- Restore a dropped database (restores all schemas + tables inside)
UNDROP DATABASE sales_db;
```

3. Restore via CLONE + SWAP (canonical restore pattern):^[9]

```
-- Step 1: Create a clone at the "good" point in time
CREATE TABLE orders_restored CLONE orders AT (OFFSET => -3600);

-- Step 2: Swap the restored table with the current "bad" table
ALTER TABLE orders SWAP WITH orders_restored;

-- Step 3: Drop the old bad table (now named orders_restored after swap)
DROP TABLE orders_restored;
```

Adjusting Time Travel Retention^[4]

```
-- Set at table creation
CREATE TABLE my_table (id INT, name VARCHAR)
  DATA_RETENTION_TIME_IN_DAYS = 30;

-- Modify existing table
ALTER TABLE my_table SET DATA_RETENTION_TIME_IN_DAYS = 7;

-- Set to 0 (disables Time Travel – data moves to Fail-Safe immediately on delete)
ALTER TABLE my_table SET DATA_RETENTION_TIME_IN_DAYS = 0;

-- Check current retention setting
SELECT TABLE_NAME, RETENTION_TIME
```

```
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'PUBLIC';
```

Edge case — reducing retention retroactively:[^4]

If you reduce retention from 10 days to 1 day, data from days 2-10 is immediately moved to Fail-Safe. It is NO LONGER accessible via Time Travel. This is a one-way door.

TOPIC 5.1.2 — Fail-Safe

☒ Layman's Explanation

Fail-Safe is Snowflake's last-resort data protection vault. After Time Travel expires, data enters Fail-Safe. Think of it as a safety deposit box that ONLY Snowflake employees can open — you cannot access the data yourself. If a catastrophic failure or corruption occurs, Snowflake's support team can recover data from Fail-Safe on your behalf.

☒ Technical Explanation^[6][5][^7]

What it is: A non-configurable 7-day data recovery period that begins AFTER the Time Travel retention period ends (for permanent tables only). During Fail-Safe, data can ONLY be recovered by Snowflake's internal support team — users have NO direct access.

Key rules:

- Fail-Safe = exactly 7 days — no more, no less — **NOT** configurable
- **ONLY** for Permanent tables — Transient and Temporary tables have ZERO Fail-Safe
- Users **CANNOT** query or access Fail-Safe data directly
- Recovery from Fail-Safe requires opening a **Snowflake support ticket** — recovery is not guaranteed and may take time
- Fail-Safe data contributes to storage costs (you pay for those 7 days of data)

Total data retention timeline for a Permanent table:



Storage cost:

- Time Travel data: charged at standard storage rates (same as active data)

- Fail-Safe data: charged at standard storage rates
- Total storage = Active Data + Time Travel Data + Fail-Safe Data
- Transient tables: ZERO extra storage cost beyond Time Travel (no Fail-Safe)
- This is why Transient tables exist — to control storage costs for staging/temporary data

TOPIC 5.1.3 — Zero-Copy Cloning

☒ Layman's Explanation

Imagine photocopying a 10,000-page book. Normal copying: takes time, uses paper, creates a full duplicate. Zero-Copy Cloning: you create a "smart bookmark" that points to the original pages. The clone instantly exists, uses NO extra storage initially. Only when you or the original make changes do new "pages" get written — you only pay for what's different.

This is incredibly powerful for:

- Creating instant dev/test environments from production data
- Making point-in-time snapshots before a risky migration
- Sharing a copy of data with a team without duplicating storage costs

☒ Technical Explanation^[10][8]

What it is: Zero-Copy Cloning creates a logical copy of a Snowflake object (table, schema, database) that initially shares the same underlying micro-partitions as the source. No data is physically duplicated — the clone is created instantly regardless of data size. Storage cost for the clone is \$0 until the clone or source diverge (new writes create new micro-partitions).

Clone behavior (micro-partition sharing):

Before any changes:

Source Table → [Partition A] [Partition B] [Partition C]

Clone Table → [Partition A] [Partition B] [Partition C]

(Same physical partitions – shared)

After source inserts new data:

Source Table → [Partition A] [Partition B] [Partition C] [NEW Partition D]

Clone Table → [Partition A] [Partition B] [Partition C]

(Clone doesn't get D – diverged. Storage cost = only Partition D for source)

After clone is modified:

Clone Table → [Partition A] [Partition B] [MODIFIED C'] [Partition C]

(Modified partition C' is new – clone pays for C'. Old C still shared with source)

What can be cloned:^[10]

- Tables (including all data + constraints + clustering keys + masking policies)

- Schemas (including all objects inside)
- Databases (including all schemas and objects inside)
- Stages (Named internal stages)
- File Formats
- Sequences
- Tasks, Streams, Pipes (in some contexts)

What CANNOT be cloned via Time Travel:[⁴]

- External Tables
- Internal (Snowflake) Stages when using AT timestamp on schema/DB clone
- Temporary Tables cannot be cloned to a Permanent table (only to Temporary or Transient)

Clone Syntax[⁸][¹⁰]

```
-- Clone a table (instant, no data copy)
CREATE TABLE orders_dev CLONE orders;

-- Clone a schema (clones all tables + objects in it)
CREATE SCHEMA sales_dev CLONE sales_prod;

-- Clone a database (clones all schemas + all tables)
CREATE DATABASE analytics_dev CLONE analytics_prod;

-- Clone from a point in the past (Time Travel + Clone = powerful restore)
CREATE TABLE orders_backup CLONE orders AT (OFFSET => -7200);
CREATE TABLE orders_before_update CLONE orders BEFORE (STATEMENT => 'query_id_here')
CREATE TABLE orders_snapshot CLONE orders AT (TIMESTAMP => '2026-01-10 09:00:00':TIMESTAMP)
```

Critical edge cases:[¹⁰][⁴]

1. **Privileges ARE inherited:** Grants on the source object are NOT automatically copied to the clone. The clone is a new object — you must grant privileges separately.
 - Exception: OWNERSHIP of the clone is granted to the role that ran CREATE ... CLONE
2. **Database clone fails if child object exceeds Time Travel window:** Cloning a DB at a timestamp 2 days ago fails if any TABLE inside has DATA_RETENTION_TIME_IN_DAYS = 1 — that table's history doesn't extend that far back.
 - Workaround: Use IGNORE TABLES WITH INSUFFICIENT DATA_RETENTION_IN_DAYS parameter
3. **Masking policies on cloned columns:** If source columns have masking policies, the clone inherits those same masking policy references — same policies apply to the clone.
4. **Transient clone of a Permanent table:**

```
CREATE TRANSIENT TABLE orders_stage CLONE orders_permanent;
-- Result: New transient table – NO Fail-Safe, max 1-day Time Travel
-- Useful for creating cost-efficient dev copies of production tables
```

5. Storage during clone:

- If you clone a 10TB table: Initial cost = \$0 (shares partitions)
- If source and clone both update: each pays for their own new partitions
- If clone is dropped: shared partitions still belong to source — no data loss

TOPIC 5.1.4 — Data Replication and Failover

☒ Layman's Explanation

Imagine your company's main office is in New York. You have all your critical data there. If New York has a disaster (power outage, flood), your business stops. **Replication** is like having a real-time copy of your entire office in Los Angeles. If New York goes down, you flip a switch and your LA office takes over — **Failover**. Snowflake replication works across clouds (AWS → Azure → GCP) and across regions.

☒ Technical Explanation^{[11][12][13]}

Replication Group: A collection of Snowflake objects (databases, shares, account objects) that are replicated as a unit from a **primary** account to one or more **secondary** accounts. Secondary replicas are **read-only**.

Failover Group: A replication group that can also perform **failover** — promoting a secondary to become the new primary, making it read-write.

What can be replicated:^[13]

- Databases (tables, views, schemas)
- Shares (data sharing configurations)
- Account objects: Users, Roles, Warehouses, Resource Monitors, Network Policies, Account Parameters
- Storage Integrations (with ETL replication enabled)
- Tasks, Pipes, Load History (with `ENABLE_ETL_REPLICATION = TRUE`)
- Workspaces (GA as of April 2026)^[14]

Replication is **NOT** real-time:

- Replication runs on a **defined schedule** (set by the user)
- Secondary accounts are point-in-time consistent as of the last refresh
- Refresh frequency = balance between RPO (Recovery Point Objective) and cost

Failover process:

1. A disaster or planned failover occurs
2. Promote a secondary account to primary: `ALTER FAILOVER GROUP ... PRIMARY`
3. The secondary becomes the new primary with read-write access
4. The old primary becomes a secondary

Failback: Re-promoting the original primary back to primary status after the disaster is resolved.

Edition requirement:[^13]

- Database replication: All editions (Standard+)
- Failover groups and account object replication: Business Critical Edition required for full functionality
- Cross-region replication: All editions (within the same region group)

☒ Hands-On Queries: Time Travel, Fail-Safe, Cloning & Replication

```
-- =====
-- SETUP
-- =====

USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS COLLAB_LAB;
USE DATABASE COLLAB_LAB;
CREATE SCHEMA IF NOT EXISTS PROTECTION_LAB;
USE SCHEMA PROTECTION_LAB;
CREATE WAREHOUSE IF NOT EXISTS collab_wh WAREHOUSE_SIZE = 'X-SMALL' AUTO_SUSPEND = 60;
USE WAREHOUSE collab_wh;

-- Create a Permanent table with 7-day Time Travel
CREATE OR REPLACE TABLE customer_orders (
    order_id    INTEGER,
    customer_id INTEGER,
    order_date   DATE,
    amount       NUMBER(12,2),
    status       VARCHAR(20)
) DATA_RETENTION_TIME_IN_DAYS = 7;

-- Create a Transient table for comparison
CREATE OR REPLACE TRANSIENT TABLE customer_orders_transient (
    order_id    INTEGER,
    customer_id INTEGER,
    amount       NUMBER(12,2)
);

-- Insert initial data
INSERT INTO customer_orders VALUES
    (1, 101, '2026-01-01', 500.00, 'COMPLETED'),
    (2, 102, '2026-01-02', 1200.00, 'COMPLETED'),
    (3, 103, '2026-01-03', 350.00, 'PENDING'),
```



```

(4, 104, '2026-01-04', 890.00, 'COMPLETED'),
(5, 105, '2026-01-05', 2200.00, 'CANCELLED');

SELECT * FROM customer_orders;

-- =====
-- PART 1: Time Travel – Querying the Past
-- =====
-- Save current timestamp (BEFORE any changes)
SET good_timestamp = CURRENT_TIMESTAMP();

-- Wait 1 second (to create a detectable time difference)
CALL SYSTEM$WAIT(1);

-- Make changes to simulate an "accident"
UPDATE customer_orders SET status = 'ERROR' WHERE order_id IN (1, 2, 3);
DELETE FROM customer_orders WHERE order_id IN (4, 5);

SELECT * FROM customer_orders; -- Only see 3 rows with ERROR status

-- TIME TRAVEL: Query data as it was at the good timestamp
SELECT * FROM customer_orders AT (TIMESTAMP => $good_timestamp);
-- Should see ALL 5 rows with original status!

-- TIME TRAVEL: Query 60 seconds in the past (OFFSET)
SELECT * FROM customer_orders AT (OFFSET => -60);
-- Should see the pre-change state

-- Get the query ID of the UPDATE (from Query History)
-- Then use BEFORE (STATEMENT) to see data before the update
-- SELECT LAST_QUERY_ID(); -- Get the update's query ID
-- SELECT * FROM customer_orders BEFORE (STATEMENT => '<update_query_id>');

-- View query history to find the DELETE query ID
SELECT QUERY_ID, QUERY_TEXT, START_TIME
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE QUERY_TEXT ILIKE '%DELETE%customer_orders%'
ORDER BY START_TIME DESC
LIMIT 5;

-- =====
-- PART 2: Time Travel – UNDROP
-- =====
-- Drop the table
DROP TABLE customer_orders;
SELECT * FROM customer_orders; -- ERROR: table does not exist

-- UNDROP it!
UNDROP TABLE customer_orders;
SELECT * FROM customer_orders; -- Table is back with its data!

-- Verify retention settings survived undrop
SELECT TABLE_NAME, RETENTION_TIME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_NAME = 'CUSTOMER_ORDERS';

```

```

-- Drop a schema and undrop it (restores all tables inside)
CREATE SCHEMA test_schema_for_drop;
CREATE TABLE test_schema_for_drop.test_tbl (id INT);
INSERT INTO test_schema_for_drop.test_tbl VALUES (1),(2),(3);

DROP SCHEMA test_schema_for_drop;

-- Restore it
UNDROP SCHEMA test_schema_for_drop;
SELECT * FROM test_schema_for_drop.test_tbl; -- Data is back!

-- Clean up test schema
DROP SCHEMA test_schema_for_drop;

-- =====
-- PART 3: Time Travel – Canonical Restore Pattern
-- =====

-- Simulate bad update (all prices set to $0 accidentally)
UPDATE customer_orders SET amount = 0;
SELECT * FROM customer_orders; -- All amounts = 0 (bad!)

-- Restore using CLONE + SWAP pattern
-- Step 1: Create a clone at the pre-update state
CREATE OR REPLACE TABLE customer_orders_restored
    CLONE customer_orders AT (OFFSET => -120); -- 2 minutes ago (before the update)

SELECT * FROM customer_orders_restored; -- Should have original amounts

-- Step 2: Swap restored with corrupted table
ALTER TABLE customer_orders SWAP WITH customer_orders_restored;

-- Step 3: Verify the swap
SELECT * FROM customer_orders; -- Original amounts restored!

-- Step 4: Drop the old corrupted table (now named customer_orders_restored)
DROP TABLE customer_orders_restored;

-- =====
-- PART 4: Zero-Copy Cloning
-- =====

-- Reset data
TRUNCATE TABLE customer_orders;
INSERT INTO customer_orders VALUES
    (1, 101, '2026-01-01', 500.00, 'COMPLETED'),
    (2, 102, '2026-01-02', 1200.00, 'COMPLETED'),
    (3, 103, '2026-01-03', 350.00, 'PENDING'),
    (4, 104, '2026-01-04', 890.00, 'COMPLETED'),
    (5, 105, '2026-01-05', 2200.00, 'CANCELLED');

-- Create a zero-copy clone (instant, $0 storage initially)
CREATE OR REPLACE TABLE customer_orders_dev CLONE customer_orders;

-- Both tables have the same data
SELECT COUNT(*) FROM customer_orders;      -- 5
SELECT COUNT(*) FROM customer_orders_dev;  -- 5

```

```

-- Modify the clone (does NOT affect source)
UPDATE customer_orders_dev SET status = 'TEST' WHERE order_id = 1;

-- Source is unchanged
SELECT order_id, status FROM customer_orders WHERE order_id = 1;      -- COMPLETED
SELECT order_id, status FROM customer_orders_dev WHERE order_id = 1;  -- TEST

-- Modify the source (does NOT affect clone)
INSERT INTO customer_orders VALUES (6, 106, CURRENT_DATE(), 999.00, 'NEW');

SELECT COUNT(*) FROM customer_orders;      -- 6 (has the new row)
SELECT COUNT(*) FROM customer_orders_dev;  -- 5 (doesn't have it)

-- Clone a Schema (all objects inside are cloned)
CREATE SCHEMA protection_lab_dev CLONE protection_lab;

-- Verify: Clone schema has the same tables
SHOW TABLES IN SCHEMA protection_lab_dev;

-- Clone a Database
CREATE DATABASE collab_lab_dev CLONE collab_lab;
SHOW SCHEMAS IN DATABASE collab_lab_dev;

-- Transient clone of a permanent table (no Fail-Safe, cost-efficient for dev)
CREATE TRANSIENT TABLE customer_orders_staging CLONE customer_orders;
SHOW TABLES LIKE 'CUSTOMER_ORDERS_STAGING';
-- Type = TRANSIENT, Retention Time = 1 (not 7)

-- Check Time Travel retention of clone vs source
SELECT TABLE_NAME, TABLE_TYPE, RETENTION_TIME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'PROTECTION_LAB'
ORDER BY TABLE_NAME;

-- =====
-- PART 5: Adjusting Time Travel Retention
-- =====

-- Set retention to 30 days (Enterprise edition required for >1 day)
ALTER TABLE customer_orders SET DATA_RETENTION_TIME_IN_DAYS = 30;

-- Disable Time Travel (data moves to Fail-Safe immediately on delete)
ALTER TABLE customer_orders_staging SET DATA_RETENTION_TIME_IN_DAYS = 0;

-- Re-enable
ALTER TABLE customer_orders_staging SET DATA_RETENTION_TIME_IN_DAYS = 1;

-- Check storage consumption (Time Travel + Fail-Safe contributes here)
USE ROLE ACCOUNTADMIN;
SELECT TABLE_NAME, ACTIVE_BYTES/1048576 AS active_mb,
       TIME_TRAVEL_BYTES/1048576 AS time_travel_mb,
       FAILSAFE_BYTES/1048576 AS failsafe_mb,
       RETAINED_FOR_CLONE_BYTES/1048576 AS clone_retained_mb
FROM SNOWFLAKE.ACCOUNT_USAGE.TABLE_STORAGE_METRICS
WHERE TABLE_CATALOG = 'COLLAB_LAB'
      AND TABLE_SCHEMA = 'PROTECTION_LAB'
ORDER BY (ACTIVE_BYTES + TIME_TRAVEL_BYTES + FAILSAFE_BYTES) DESC;

```

```

-- =====
-- PART 6: Data Replication (view account-level replication objects)
-- =====
-- Check replication accounts available in your organization
SHOW REPLICATION ACCOUNTS;

-- Show existing replication groups (if any)
SHOW REPLICATION GROUPS;

-- Show failover groups (if any)
SHOW FAILOVER GROUPS;

-- Create a replication group (requires another account in your org)
-- CREATE REPLICATION GROUP my_replication_group
--     OBJECT_TYPES = DATABASES
--     ALLOWED_DATABASES = COLLAB_LAB
--     ALLOWED_ACCOUNTS = <org_name>.<target_account_name>
--     REPLICATION_SCHEDULE = '10 MINUTE';

-- Cleanup
USE ROLE SYSADMIN;
DROP DATABASE IF EXISTS collab_lab_dev;
DROP SCHEMA IF EXISTS protection_lab_dev;

```

☒ Topic 5.1 — 20 Practice Questions: Time Travel, Fail-Safe, Cloning & Replication

Q1. What is the MAXIMUM Time Travel retention period for a Permanent table on Snowflake Enterprise Edition?

- A) 1 day
- B) 14 days
- C) 30 days
- D) 90 days

Q2. A developer accidentally deletes all rows from a Transient table. The table's Time Travel retention is set to 1 day. Can the rows be recovered from Fail-Safe after the 1-day window expires?

- A) Yes — Transient tables have a 7-day Fail-Safe just like Permanent tables
- B) Yes — Fail-Safe for Transient tables is 1 day
- C) No — Transient tables have NO Fail-Safe period (0 days). Once Time Travel expires, the data is permanently gone.
- D) No — Transient tables cannot use Time Travel or Fail-Safe at all

Q3. Which Time Travel clause queries the data as it existed BEFORE a specific SQL statement executed?

- A) AT (TIMESTAMP => ...)

- B) AT (OFFSET => ...)
- C) BEFORE (STATEMENT => 'query_id')
- D) AT (STATEMENT => 'query_id')

Q4. What does `UNDROP TABLE orders` do?

- A) Permanently deletes the table, bypassing Time Travel
- B) Restores the most recently dropped version of the table within the Time Travel retention window
- C) Restores the table from Fail-Safe storage
- D) Creates a new empty table named `orders` to replace the dropped one

Q5. A Permanent table has `DATA_RETENTION_TIME_IN_DAYS = 7`. It is dropped today. What happens to the data?

- A) Data is permanently deleted after 7 days (no Fail-Safe)
- B) Data is accessible via Time Travel for 7 days, then enters Fail-Safe for an additional 7 days — total 14 days before permanent deletion
- C) Data enters Fail-Safe immediately after the table is dropped
- D) Data is backed up to the assigned cloud storage bucket

Q6. Zero-Copy Cloning creates a clone of a 10TB table. How much additional storage is consumed immediately after the clone is created?

- A) 10TB — a full physical copy is made
- B) 5TB — Snowflake compresses the clone by 50%
- C) 0TB — the clone shares the same underlying micro-partitions with no physical duplication
- D) Storage equivalent to one day of data change (delta)

Q7. Which of the following objects CAN be cloned in Snowflake? (*Select ALL that apply*)

- A) Permanent tables
- B) Schemas (including all objects within)
- C) Databases
- D) External Tables
- E) Transient Tables

Q8. A developer creates a Transient clone of a Permanent table. What are the Time Travel and Fail-Safe properties of the cloned Transient table?

- A) It inherits the Permanent table's retention settings — 90-day Time Travel, 7-day Fail-Safe
- B) It becomes a Transient table — max 1-day Time Travel, 0-day Fail-Safe (NO Fail-Safe)

- C) It becomes a Temporary table — exists only for the current session
- D) It inherits Time Travel from the source but has no Fail-Safe (Transient rules)

Q9. What is Fail-Safe in Snowflake?

- A) A user-configurable backup period where you can query historical data via SQL
- B) A non-configurable 7-day data protection period for Permanent tables, AFTER Time Travel ends, accessible ONLY by Snowflake Support — not by users directly
- C) An automatic replication mechanism that copies data to another region in real-time
- D) A checkpoint system that saves snapshots of tables every hour

Q10. An ACCOUNTADMIN reduces DATA_RETENTION_TIME_IN_DAYS from 10 to 1 for a table. What happens to historical data from days 2 through 10?

- A) Nothing — the data remains in Time Travel until the original 10-day window expires
- B) Historical data from days 2-10 is immediately moved to Fail-Safe — no longer accessible via Time Travel queries
- C) Historical data from days 2-10 is permanently and immediately deleted
- D) The change takes effect for future data only — existing historical data keeps its original 10-day window

Q11. What is the canonical (recommended) approach to restore a table that had a bad UPDATE applied?

- A) Run `UNDROP TABLE` — it automatically rolls back the last update
- B) Create a new clone from a pre-update timestamp with `CLONE ... AT (OFFSET => ...)`, then `ALTER TABLE ... SWAP WITH` to replace the corrupted table
- C) Run `RESTORE TABLE ... FROM FAILSAFE` to pull from the 7-day Fail-Safe backup
- D) Use `MERGE INTO` with a Time Travel query as the source to re-apply original values

Q12. What does `AT (OFFSET => -3600)` mean in a Time Travel query?

- A) Query data 3600 milliseconds (3.6 seconds) in the past
- B) Query data 3600 minutes in the past
- C) Query data 3600 seconds (1 hour) in the past
- D) Query data at version number 3600

Q13. When you clone a database, do the privilege grants on the source tables automatically transfer to the clone?

- A) Yes — all privileges are automatically inherited by the clone
- B) No — privilege grants are NOT copied to cloned objects. The clone is a new object owned by the role that ran the `CLONE` command.
- C) Only `OWNERSHIP` and `SELECT` privileges are inherited

- D) Yes, but only if the clone is created by ACCOUNTADMIN

Q14. A Snowflake Standard Edition account has a Permanent table. What is the maximum Time Travel retention period it can configure?

- A) 90 days
- B) 30 days
- C) 14 days
- D) 1 day

Q15. A company clones its production database to a dev database. A week later, they update many rows in the production database. What happens to the dev clone's storage cost?

- A) Dev clone's storage doubles — Snowflake now maintains two full copies
- B) Dev clone's storage stays near \$0 — it still shares unchanged micro-partitions with production. Only new/changed production partitions use additional storage.
- C) Dev clone's storage increases proportionally to the production changes
- D) Dev clone automatically receives the production updates (shared storage = shared changes)

Q16. What is the MINIMUM Time Travel retention period that can be set on a Transient table?

- A) 0 days
- B) 1 day
- C) 7 days
- D) Transient tables cannot have Time Travel

Q17. Which Snowflake feature enables copying account objects (users, roles, warehouses, network policies) across regions for disaster recovery purposes?

- A) Zero-Copy Cloning
- B) Secure Data Sharing
- C) Replication Groups / Failover Groups
- D) Time Travel

Q18. When a Failover Group's secondary account is **promoted to primary**, what changes?

- A) The secondary becomes read-write (primary); the original primary becomes read-only (secondary)
- B) The secondary is permanently deleted and the primary remains active
- C) Both accounts become co-primary with simultaneous read-write access
- D) The secondary remains read-only; only account metadata is promoted

Q19. What storage costs are incurred by Snowflake Fail-Safe data?

- A) No charge — Fail-Safe is included free as part of the Snowflake license
- B) Charged at standard Snowflake storage rates — Fail-Safe bytes appear in `TABLE_STORAGE_METRICS`
- C) Charged at a premium rate (2× standard storage) due to the recovery infrastructure overhead
- D) Charged only if Snowflake Support actually performs a data recovery

Q20. A developer runs: `CREATE TABLE dev_clone CLONE prod_table AT (OFFSET => -172800)`. The `prod_table` has `DATA_RETENTION_TIME_IN_DAYS = 1`. What happens?

- A) The clone succeeds — `OFFSET` is measured in hours, so $172800 = 1 \text{ day} = \text{within retention}$
- B) The clone FAILS — `OFFSET` is in seconds, so $172800 \text{ seconds} = 48 \text{ hours} = 2 \text{ days ago}$, which is beyond the 1-day Time Travel retention period
- C) The clone succeeds using the last available snapshot at the 1-day boundary
- D) The clone succeeds but `dev_clone` contains empty data from that historical point

SUBDOMAIN 5.2 — Snowflake's Data Sharing Capabilities

☒ Layman's Explanation

Imagine you have a spreadsheet of live customer data. Your business partner needs to see it. You could email it to them (a copy of the data at that moment). But what if they need the LIVE version, updated every second, without you sending a new email every time?

Secure Data Sharing is Snowflake's answer: you share a live, read-only window into YOUR data directly with another Snowflake account. No data is copied. No files are sent. The consumer queries YOUR data in YOUR storage, in real-time — they just can't see the raw data without your permission, and they cannot modify it.

Key players:

- **Provider:** YOU — the one who owns and shares the data
- **Consumer:** The other Snowflake account reading your data
- **Share Object:** The "package" you create defining what data to share
- **Reader Account:** A free mini-Snowflake account you create for consumers who don't have Snowflake

☒ Technical Explanation

Core Architecture of Secure Data Sharing^{[15][16][^17]}

How it works technically:

1. Provider creates a **SHARE** object (no data is moved — it's metadata)
2. Provider grants database objects (tables, views, secure views) to the SHARE
3. Provider adds one or more consumer account identifiers to the SHARE
4. Consumer creates a read-only DATABASE from the SHARE
5. Consumer queries the data — their compute runs against provider's storage
6. **No data is copied** — zero storage overhead for consumers
7. Data updates in the provider's source are immediately visible to consumers

Key characteristics:

- **Real-time:** Consumers see live data (new/modified rows appear immediately after provider commits)
- **No data copy:** Provider pays for storage; consumer pays only for their query compute
- **Read-only for consumer:** Consumer CANNOT insert, update, delete, or use Time Travel/Fail-Safe on shared data
- **Cross-region:** Data sharing IS supported across regions and clouds (since Snowflake manages the data layer globally)

Sharing Methods — Direct Share vs. Listing^{[18][19][^15]}

Method	Description	Visibility	Best For
Direct Share	Share directly to specific account identifiers	Private (only specified accounts)	Partner-to-partner sharing within same region; ad-hoc sharing
Listing (Private)	Wrap a share in a Listing with metadata (title, description, sample queries)	Private (specific accounts)	Enhanced sharing with documentation across any region
Listing (Public — Snowflake Marketplace)	Publish to the Snowflake Marketplace for any Snowflake user to discover and request	Public (all Snowflake users)	Data products, monetization, broad distribution
Data Exchange	Private marketplace for a controlled group of accounts (e.g., a consortium)	Semi-private (defined group)	Industry data exchanges, internal org sharing
Data Clean Room	Controlled sharing where provider limits which queries can run	Highly restricted	Privacy-preserving analytics (PII-sensitive collaboration)

Creating and Managing Shares — SQL Reference^{[16][17]}

```
-- Step 1: Create the SHARE object (ACCOUNTADMIN required)
USE ROLE ACCOUNTADMIN;

CREATE SHARE sales_data_share
    COMMENT = 'Sharing monthly sales data with partner';

-- Step 2: Grant access to the database (must grant before individual objects)
GRANT USAGE ON DATABASE sales_db TO SHARE sales_data_share;

-- Step 3: Grant access to the schema
GRANT USAGE ON SCHEMA sales_db.public TO SHARE sales_data_share;

-- Step 4: Grant access to specific tables or secure views
GRANT SELECT ON TABLE sales_db.public.monthly_sales TO SHARE sales_data_share;
GRANT SELECT ON VIEW sales_db.public.sales_summary_secure TO SHARE sales_data_share;

-- Adding a SECURE VIEW (best practice – hides underlying logic from consumers)
CREATE SECURE VIEW sales_summary_secure AS
    SELECT region, SUM(amount) AS total_revenue
    FROM monthly_sales
    GROUP BY region;
-- Without SECURE: consumers can see the view's SQL definition via SHOW VIEWS

-- Step 5: Add consumer accounts to the share
ALTER SHARE sales_data_share ADD ACCOUNTS = consumer_org.consumer_account1;
ALTER SHARE sales_data_share ADD ACCOUNTS = consumer_org.consumer_account2;
-- Can add multiple consumers – data is visible to all added accounts

-- Remove a consumer
ALTER SHARE sales_data_share REMOVE ACCOUNTS = consumer_org.consumer_account2;

-- View share details
SHOW SHARES;
DESC SHARE sales_data_share;

-- =====
-- On the CONSUMER side:
-- =====

-- Create a read-only database from the share
CREATE DATABASE partner_sales_db FROM SHARE provider_org.provider_account.sales_data;

-- Grant access to users in the consumer account
GRANT IMPORTED PRIVILEGES ON DATABASE partner_sales_db TO ROLE data_analysts;

-- Query the shared data (consumer's compute, provider's storage)
USE DATABASE partner_sales_db;
SELECT * FROM public.monthly_sales ORDER BY sale_month DESC;
```

Objects That CAN and CANNOT Be Shared^[17]^[16]

Object Type	Can Be Shared?	Notes
Tables	✔ Yes	Direct SELECT grants
Secure Views	✔ Yes	Best practice — hides definition
External Tables	✔ Yes	Share metadata, consumer reads their cloud files
Secure Materialized Views	✔ Yes	Enterprise+
Regular (Non-Secure) Views	✗ No	Consumer would see the view definition — use SECURE VIEW
Stored Procedures	✗ No	Cannot be shared
UDFs (standard)	✗ No	Must use SECURE UDFs or share via Native App
Stages	✗ No	Cannot be directly shared

Secure Views — Why They Matter^[^16]

Non-Secure View: The view's SQL definition is visible to anyone who can SHOW VIEWS — including consumers of a share. This can expose business logic, table structures, or join patterns.

Secure View: The view's definition is hidden from all users except the owner. Required for sharing because Snowflake's optimizer also changes behavior: it does NOT push predicates from the consumer's query into the view's SQL — ensuring the provider's logic is always respected.

```
-- Non-secure view (NOT shareable):
CREATE VIEW sales_view AS SELECT * FROM internal_sales WHERE region != 'Confidential'
-- A consumer could bypass this: Snowflake might optimize the WHERE clause away!

-- Secure view (shareable, definition hidden, predicate pushdown disabled):
CREATE SECURE VIEW sales_view_secure AS
    SELECT * FROM internal_sales WHERE region != 'Confidential';
-- Definition is HIDDEN; predicate cannot be bypassed
```

Reader Accounts — Sharing with Non-Snowflake Users^[20]^[15]

What they are: Mini Snowflake accounts created BY the provider for consumers who do NOT have their own Snowflake account. The reader account is entirely owned and managed by the provider.

Key rules:

- Reader account can ONLY consume data from the provider account that created it
- Reader account users CANNOT load their own data, create their own databases, or do DML

- Reader account compute is billed to the PROVIDER (the provider owns the warehouses in the reader account)
- A provider can create a MAXIMUM of 20 reader accounts
- Reader accounts are like a "read-only portal" into the provider's shared data

```
-- Create a reader account (ACCOUNTADMIN required)
CREATE MANAGED ACCOUNT my_partner_reader_account
  ADMIN_NAME = 'partner_admin'
  ADMIN_PASSWORD = 'SecurePassword123!'
  TYPE = READER
  COMMENT = 'Reader account for Partner Corp';

-- View created reader accounts
SHOW MANAGED ACCOUNTS;

-- Share data with the reader account
ALTER SHARE sales_data_share ADD ACCOUNTS = my_partner_reader_account;
```

Snowflake Marketplace and Listings^[19][18]

Snowflake Marketplace: A public catalog where providers publish **Listings** — data products that any Snowflake user can discover and request access to.

Listing vs. Direct Share:

Feature	Direct Share	Listing
Metadata (title, description)	✗ None	✓ Rich metadata
Cross-region sharing	Limited (same region)	✓ Any region, any cloud
Public discoverability	✗ Private only	✓ Can be public on Marketplace
Usage monitoring	✗ Basic	✓ Detailed consumer usage analytics
Monetization	✗ Not available	✓ Paid listings supported
Sample queries	✗ Not available	✓ Can include sample queries

Creating a Listing (via Provider Studio in Snowsight):

- Navigate to Data Products → Provider Studio → + Listing
- Select the share to package as a listing
- Add title, description, categories, sample queries
- Choose: Private (specific accounts) or Public (Marketplace)
- Publish

Resharing (Cross-Sharing)^{[15][16]}

A consumer can re-share data they received from a provider to ANOTHER Snowflake account — this is called **resharing**.

Resharing rules:

- The provider must EXPLICITLY grant the consumer permission to reshare
- By default, consumers CANNOT reshare received data
- To allow: ALTER SHARE my_share SET ALLOW_RESHARING = TRUE (provider action)
- Used in multi-tier distribution chains (e.g., data aggregator → data provider → end consumer)

☒ Hands-On Queries: Secure Data Sharing

```
-- =====
-- SETUP – Provider Side
-- =====

USE ROLE ACCOUNTADMIN;
USE DATABASE COLLAB_LAB;
CREATE SCHEMA IF NOT EXISTS SHARING_LAB;
USE SCHEMA SHARING_LAB;
USE WAREHOUSE collab_wh;

-- Create tables to share
CREATE OR REPLACE TABLE product_catalog (
    product_id    INTEGER,
    product_name  VARCHAR(100),
    category      VARCHAR(50),
    list_price    NUMBER(10,2),
    available     BOOLEAN
);

CREATE OR REPLACE TABLE public_pricing (
    product_id    INTEGER,
    region        VARCHAR(30),
    sale_price    NUMBER(10,2),
    effective_date DATE
);

-- Internal sensitive table (NOT shared)
CREATE OR REPLACE TABLE internal_cost_data (
    product_id    INTEGER,
    cost          NUMBER(10,2),    -- NEVER share this!
    supplier      VARCHAR(100)
);

-- Insert sample data
INSERT INTO product_catalog VALUES
    (1, 'Widget Pro',    'Electronics', 299.99, TRUE),
    (2, 'Gizmo Basic',   'Electronics', 49.99,  TRUE),
    (3, 'SuperGadget',   'Tools',      149.99, FALSE),
```

```

(4, 'DataBox',      'Software',      599.00, TRUE);

INSERT INTO public_pricing VALUES
  (1, 'North', 279.99, '2026-01-01'),
  (1, 'South', 289.99, '2026-01-01'),
  (2, 'North', 44.99,  '2026-01-01'),
  (2, 'South', 44.99,  '2026-01-01');

INSERT INTO internal_cost_data VALUES
  (1, 120.00, 'Supplier A'),
  (2, 18.00,  'Supplier B');

-- =====
-- PART 1: Create a Secure View (to hide sensitive data)
-- =====
-- Non-secure view (definition visible – do NOT use for sharing)
CREATE OR REPLACE VIEW product_catalog_view AS
  SELECT product_id, product_name, category, list_price
  FROM product_catalog
  WHERE available = TRUE;

-- Secure view (definition HIDDEN – safe for sharing)
CREATE OR REPLACE SECURE VIEW product_catalog_secure_view AS
  SELECT product_id, product_name, category, list_price
  FROM product_catalog
  WHERE available = TRUE;

-- Verify: Non-secure view definition is visible
SHOW VIEWS LIKE 'PRODUCT_CATALOG_VIEW';
-- text column shows the full SQL definition

-- Secure view definition is hidden (text column is blank)
SHOW VIEWS LIKE 'PRODUCT_CATALOG_SECURE_VIEW';
-- text column is empty – definition is protected

-- =====
-- PART 2: Create a Share Object (Provider actions)
-- =====
-- Create the share
CREATE OR REPLACE SHARE product_data_share
  COMMENT = 'Public product catalog and pricing data for partner accounts';

-- Grant USAGE on the database and schema
GRANT USAGE ON DATABASE COLLAB_LAB TO SHARE product_data_share;
GRANT USAGE ON SCHEMA COLLAB_LAB.SHARING_LAB TO SHARE product_data_share;

-- Grant SELECT on specific objects to share
GRANT SELECT ON TABLE COLLAB_LAB.SHARING_LAB.PRODUCT_CATALOG TO SHARE product_data_s
GRANT SELECT ON TABLE COLLAB_LAB.SHARING_LAB.PUBLIC_PRICING TO SHARE product_data_sh
GRANT SELECT ON VIEW COLLAB_LAB.SHARING_LAB.PRODUCT_CATALOG_SECURE_VIEW TO SHARE pro
-- NOTE: internal_cost_data is NOT granted – it's private!

-- View the share configuration
DESC SHARE product_data_share;
SHOW SHARES;

```

```

-- =====
-- PART 3: Add Consumer Accounts (you'd need a real consumer account)
-- =====
-- In a real scenario:
-- ALTER SHARE product_data_share ADD ACCOUNTS = partner_org.partner_account;
-- The consumer would then run:
-- CREATE DATABASE partner_product_db FROM SHARE your_org.your_account.product_data_

-- Simulate what consumer would see (using same account for demo):
-- Granted to yourself to simulate consumer behavior:
-- GRANT IMPORTED PRIVILEGES ON DATABASE partner_product_db TO ROLE SYSADMIN;

-- =====
-- PART 4: Reader Account (for non-Snowflake consumers)
-- =====
-- Create a reader account (note: requires actual org setup; demo shows syntax)
-- CREATE MANAGED ACCOUNT demo_reader_account
--     ADMIN_NAME = 'reader_admin'
--     ADMIN_PASSWORD = 'SecurePassword1!'
--     TYPE = READER;

-- View managed accounts
SHOW MANAGED ACCOUNTS;

-- =====
-- PART 5: Manage and Monitor the Share
-- =====
-- View all shares in the account
SHOW SHARES;
-- Look for: kind (OUTBOUND for provider, INBOUND for consumer), to (consumer account)

-- Remove a consumer from a share
-- ALTER SHARE product_data_share REMOVE ACCOUNTS = partner_org.partner_account;

-- Revoke access to a specific object from the share
-- REVOKE SELECT ON TABLE COLLAB_LAB.SHARING_LAB.PUBLIC_PRICING FROM SHARE product_data_

-- Add a new table to an existing share
GRANT SELECT ON TABLE COLLAB_LAB.SHARING_LAB.PRODUCT_CATALOG TO SHARE product_data_s

-- Monitor data sharing usage
USE ROLE ACCOUNTADMIN;
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.DATA_SHARING_USAGE_HISTORY
WHERE START_TIME >= DATEADD('day', -30, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC
LIMIT 20;

-- =====
-- PART 6: Incoming Shares (Consumer side – Snowflake Marketplace)
-- =====
-- View available inbound shares (from Snowflake Marketplace or direct shares)
SHOW SHARES;
-- Look for KIND = INBOUND shares

-- List objects available in an inbound share

```

```
-- DESC SHARE provider_org.provider_account.some_share_name;

-- Create a database from a Marketplace share
-- CREATE DATABASE free_weather_data FROM SHARE marketplace_org.marketplace_account.
-- SELECT * FROM free_weather_data.schema_name.weather_table;

-- =====
-- Cleanup
-- =====

DROP SHARE IF EXISTS product_data_share;
DROP TABLE IF EXISTS product_catalog;
DROP TABLE IF EXISTS public_pricing;
DROP TABLE IF EXISTS internal_cost_data;
DROP VIEW IF EXISTS product_catalog_view;
DROP VIEW IF EXISTS product_catalog_secure_view;
DROP DATABASE IF EXISTS collab_lab;
```

☒ Topic 5.2 — 20 Practice Questions: Secure Data Sharing

Q1. In Snowflake's Secure Data Sharing, when a consumer queries a shared table, where is the data stored?

- A) The data is copied to the consumer's account storage when the share is created
- B) The data remains in the PROVIDER's storage — the consumer's compute queries it directly via the share
- C) The data is cached in Snowflake's Cloud Services layer for performance
- D) The data is stored in a neutral third-party Snowflake-managed location

Q2. Which role is required to CREATE and manage SHARES in Snowflake?

- A) SYSADMIN
- B) SECURITYADMIN
- C) ACCOUNTADMIN
- D) USERADMIN

Q3. A provider wants to share a database object with a consumer but doesn't want the consumer to see the SQL logic behind the view. Which object type should the provider use?

- A) Regular (non-secure) VIEW
- B) SECURE VIEW
- C) MATERIALIZED VIEW
- D) EXTERNAL TABLE

Q4. What happens to data in a shared table when the provider adds new rows?

- A) The consumer must refresh the shared database to see new rows

- B) New rows are visible to the consumer immediately after the provider commits the INSERT — sharing is always real-time live data
- C) New rows are visible after the next scheduled replication cycle (usually hourly)
- D) New rows are visible only after the provider explicitly updates the share definition

Q5. What is a Reader Account in Snowflake?

- A) A read-only internal role that can query all tables in an account
- B) A mini Snowflake account created by a data PROVIDER for consumers who do not have their own Snowflake account — owned and billed to the provider
- C) A special account type used by Snowflake Support for read-only auditing
- D) A consumer account that has been restricted to read-only access via a network policy

Q6. How many Reader Accounts can a single Snowflake provider create?

- A) 5
- B) 10
- C) 20
- D) Unlimited

Q7. A consumer receives data via a Secure Data Share. Which operations can the consumer perform on shared data? (*Select ALL that apply*)

- A) SELECT queries on shared tables and views
- B) INSERT new rows into shared tables
- C) Create views on top of the shared tables within the consumer's own database
- D) DELETE rows from shared tables
- E) Use the shared data as a source in JOIN queries with consumer's own tables

Q8. What is the Snowflake Marketplace?

- A) An internal Snowflake tool for managing warehouse billing and credits
- B) A public catalog where data providers publish Listings (data products) that any Snowflake customer worldwide can discover, request, and consume
- C) A staging area for loading external data files before COPY INTO
- D) A Snowflake-managed data warehouse for storing backup data

Q9. What is the key difference between a Direct Share and a Listing in Snowflake?

- A) Direct Shares copy data; Listings only share metadata
- B) Direct Shares can only share within the same region; Listings can share across all regions and clouds, include rich metadata, and can be published publicly on the Snowflake Marketplace

- C) Listings require a virtual warehouse to function; Direct Shares are serverless
- D) Direct Shares can be used across clouds; Listings are limited to the same cloud provider

Q10. If a provider drops and recreates a TABLE that is already included in a SHARE (even with the same name), what happens?

- A) The consumer automatically sees the new table — same name = same share grant
- B) The consumer loses access — the recreated table is treated as a NEW object and must be re-granted to the share
- C) The share is automatically updated to reference the new table
- D) Snowflake locks the share to prevent recreation of shared objects

Q11. Which of the following can a consumer do when they receive a Direct Share? (*Select ALL that apply*)

- A) Query the shared tables in real-time using their own virtual warehouse
- B) Grant access to the shared database to other roles WITHIN their own account
- C) Use Time Travel on the shared tables to query historical data
- D) Re-share the data to another account without the provider's permission
- E) Join shared data with their own local tables in the same query

Q12. A company wants to share sales data with 500 potential partner companies, many of whom do not have Snowflake accounts. What is the BEST approach?

- A) Create 500 separate Direct Shares — one per partner
- B) Create a public Listing on the Snowflake Marketplace; non-Snowflake partners can create free trial accounts to access it
- C) Use Reader Accounts for the non-Snowflake partners and a Direct Share for Snowflake partners — Reader Accounts are FREE and unlimited
- D) Create 500 Reader Accounts — one per partner (Reader Accounts are limited to 20 per provider)

Q13. Consumer compute costs in Snowflake data sharing are billed to which party?

- A) The PROVIDER — they pay for all query compute on shared data
- B) The CONSUMER — they use their own virtual warehouse to query shared data (provider pays only for storage)
- C) Snowflake charges a shared flat fee equally to both provider and consumer
- D) Neither party — query compute on shared data is free as part of Snowflake's cloud infrastructure

Q14. Why does a Secure View prevent potential data leakage compared to a regular view?

- A) Secure Views encrypt data at the column level before sharing

- B) Secure Views hide the view's SQL definition from consumers AND disable optimizer predicate pushdown — preventing consumers from potentially bypassing WHERE clause filters
- C) Secure Views automatically apply row access policies to filter out sensitive rows
- D) Secure Views apply Dynamic Data Masking to all columns in the view

Q15. What does `GRANT IMPORTED PRIVILEGES ON DATABASE partner_db TO ROLE analysts` allow on the consumer side?

- A) Allows the analysts role to import new data into the shared database
- B) Grants the analysts role the ability to query all objects in the shared database that have been shared by the provider
- C) Allows analysts to modify the share configuration
- D) Grants analysts the ability to export shared data to an external stage

Q16. A data provider shares a table with a consumer. The provider then adds a new column to the shared table. What does the consumer see?

- A) The new column immediately appears in the consumer's queries — sharing is live
- B) The consumer must recreate their database from the share to see the new column
- C) The new column is hidden until the provider re-grants SELECT on the table
- D) The share becomes invalid when schema changes are made to shared objects

Q17. Can a data consumer use Snowflake's Time Travel feature to query historical versions of shared data?

- A) Yes — consumers inherit the provider's Time Travel retention settings
- B) No — consumers cannot use Time Travel or Fail-Safe on shared databases; they have read-only access to the live current data
- C) Yes, but only up to 1 day (regardless of the provider's retention settings)
- D) Yes — consumers can access Time Travel data if they hold the IMPORTED PRIVILEGES grant

Q18. Which of the following statements about Snowflake data sharing are TRUE? (*Select ALL that apply*)

- A) No data is physically copied — sharing works via metadata references to the provider's storage
- B) Direct Share is limited to sharing within the same region and cloud
- C) Consumers must have their own Snowflake account to use Direct Share or Listings
- D) Reader Accounts are billed to the PROVIDER for compute and storage
- E) Data changes made by the provider are immediately visible to consumers

Q19. A Snowflake provider wants to allow a consumer to reshare the data with their own customers. What must the provider do?

- A) Nothing — resharing is permitted by default for all shares
- B) Create a separate share for each downstream consumer manually
- C) Enable resharing on the share by setting `ALLOW_RESHARING = TRUE` on the share — consumers cannot reshare by default
- D) The consumer must create a Reader Account to enable downstream sharing

Q20. What is a Data Clean Room in Snowflake?

- A) A compliance tool that automatically masks PII data in all tables
- B) A restricted data sharing environment where the provider controls which specific queries can be run against shared data — enabling privacy-preserving collaborative analytics
- C) A scheduled job that removes duplicate and erroneous data from shared tables
- D) A neutral Snowflake-managed storage location where two parties can upload and compare data

☒ COMPLETE ANSWER KEY — Domain 5

Topic 5.1 — Time Travel, Fail-Safe, Cloning & Replication

Q	Answer	Key Reason
1	D	90 days is the maximum Time Travel retention for Permanent tables on Enterprise Edition and higher. ^{[7][4]}
2	C	Transient tables have ZERO Fail-Safe days. After the 1-day Time Travel window closes, the data is permanently unrecoverable — no Fail-Safe exists for transient/temporary tables. ^{[5][6]}
3	C	<code>BEFORE (STATEMENT => 'query_id')</code> returns data as it existed BEFORE the specified query ran. <code>AT (STATEMENT)</code> returns data immediately after the query executed. ^[^4]
4	B	<code>UNDROP TABLE</code> restores the most recently dropped version of a table within its Time Travel retention window. The data, constraints, and retention settings are all restored. ^[^4]
5	B	A dropped Permanent table is accessible via Time Travel for the retention period (7 days), then enters Fail-Safe for 7 more days. Total: up to 14 days before permanent deletion (with 1-day default retention on Standard). ^{[5][7]}
6	C	Zero-Copy Cloning shares the same underlying micro-partitions — no physical data is duplicated at creation time. Storage cost = \$0 until changes diverge the source and clone. ^{[8][10]}
7	A, B, C, E	Permanent tables ✓, Schemas ✓, Databases ✓, Transient Tables ✓. External Tables ✗ — cannot be cloned via Time Travel (though basic structure can be cloned without history). ^{[10][4]}
8	B	When you create a TRANSIENT clone, the new table is Transient — it takes on Transient rules: max 1-day Time Travel, ZERO Fail-Safe. It does NOT inherit the source's retention settings. ^{[5][6]}

Q	Answer	Key Reason
9	B	Fail-Safe is a non-configurable 7-day period AFTER Time Travel for Permanent tables, accessible ONLY by Snowflake Support — not by users. It's a last-resort disaster recovery mechanism. ^{[5][7]}
10	B	Reducing retention retroactively moves data from the dropped days into Fail-Safe immediately — it's no longer accessible via Time Travel queries. ^[^4]
11	B	The canonical restore pattern: <code>CLONE . . . AT (OFFSET => . . .)</code> to create a pre-change copy, then <code>ALTER TABLE . . . SWAP WITH</code> to replace the corrupted table with the restored one. ^[^9]
12	C	OFFSET is in seconds. $-3600 = 3600 \text{ seconds} = 1 \text{ hour in the past}$. ^{[4][8]}
13	B	Privilege grants are NOT automatically copied to clones. The clone is treated as a new object — you must explicitly grant privileges. The cloning role receives OWNERSHIP. ^[^10]
14	D	Snowflake Standard Edition limits Permanent tables to 0 or 1 day of Time Travel (default: 1 day). 90-day retention requires Enterprise Edition. ^{[5][7]}
15	B	Zero-Copy Cloning shares unchanged micro-partitions. When production adds/modifies data, only those new partitions are charged to production. The dev clone continues sharing the original unchanged partitions at near-zero cost. ^{[8][10]}
16	A	Transient tables can be set to 0 days (disables Time Travel entirely) — 0 is the minimum. The minimum that keeps Time Travel active is 1 day. ^{[5][6]}
17	C	Replication Groups and Failover Groups replicate account objects (users, roles, warehouses, network policies, databases, shares) across regions/clouds for disaster recovery. ^{[11][12][^13]}
18	A	Failover promotes the secondary to primary (read-write) and demotes the original primary to secondary (read-only). The roles flip. ^{[11][13]}
19	B	Fail-Safe data is billed at standard Snowflake storage rates. It appears in <code>TABLE_STORAGE_METRICS . FAILSAFE_BYTES</code> . There is no discount — you pay for what's retained in Fail-Safe. ^[^5]
20	B	OFFSET is in SECONDS. $172800 \text{ seconds} = 48 \text{ hours} = 2 \text{ days ago}$. The table's Time Travel is only 1 day (86400 seconds). The clone FAILS because the point in time is beyond the retention window. ^{[8][4]}

Topic 5.2 — Secure Data Sharing

Q	Answer	Key Reason
1	B	Data NEVER leaves the provider's storage. The consumer's compute runs queries against the provider's micro-partitions directly — no copying, no movement. ^{[15][16]}
2	C	ACCOUNTADMIN is required to create, manage, and configure SHARE objects. ^{[16][17]}
3	B	SECURE VIEW hides the SQL definition from consumers. Regular views expose their definition — dangerous for sharing proprietary business logic. ^[^16]
4	B	Sharing is live — new/modified rows are immediately visible to consumers who already have access (no manual refresh needed). ^{[15][16]}
5	B	A Reader Account is a Snowflake account created by the provider for non-Snowflake consumers. It is owned and fully funded (billed) by the provider. ^{[15][20]}

Q	Answer	Key Reason
6	C	A provider can create a maximum of 20 Reader Accounts.[¹⁵ [20]
7	A, C, E	Consumers can SELECT ✓, create views on shared tables ✓, and join shared data with their own tables ✓. Consumers CANNOT INSERT ✗, DELETE ✗ — shared databases are read-only.[¹⁵
8	B	The Snowflake Marketplace is a public catalog where providers publish Listings that any Snowflake customer can discover and access.[¹⁸ [19]
9	B	Direct Shares are generally scoped to the same region; Listings support cross-region/cloud sharing, include rich metadata, and enable public Marketplace publishing.[¹⁸ [19]
10	B	A dropped and recreated object — even with the same name — is treated as a NEW object by Snowflake. The share grant is associated with the original object's ID, not just the name. The new object must be re-granted.[¹⁶ [17]
11	A, B, E	Query in real-time ✓; grant IMPORTED PRIVILEGES to roles within their account ✓; join with local tables ✓. Time Travel on shared data ✗ (read-only access only); resharing without permission ✗. [¹⁵ [16]
12	B	For mass distribution including non-Snowflake users, a public Listing on the Marketplace is most efficient. Reader Accounts are capped at 20 — nowhere near 500.[¹⁵ [18]
13	B	Consumer pays for their own query compute (their virtual warehouse). Provider pays for storage. This is one of the most exam-tested facts about Snowflake sharing.[¹⁵ [16]
14	B	Secure Views hide the definition AND disable optimizer predicate pushdown — preventing consumers from issuing queries that could bypass the view's WHERE conditions to infer protected data.[¹⁶
15	B	GRANT IMPORTED PRIVILEGES ON DATABASE allows a role in the consumer account to query all objects in the shared database that were granted by the provider.[¹⁵
16	A	Schema changes (new columns, etc.) to shared tables are immediately reflected for consumers — the share is a live window, not a snapshot.[¹⁵ [16]
17	B	Consumers CANNOT use Time Travel on shared databases. They have read-only access to the current live data only — no historical access via AT/BEFORE clauses.[¹⁵
18	A, B, D, E	No data copy ✓; Direct Share is region-limited ✓; Reader Accounts billed to provider ✓; changes immediately visible ✓. Consumers DO need their own Snowflake account for Direct Share (Reader Accounts are the solution for non-Snowflake users) — so C is partially true but misleading ✗.[¹⁵ [16][¹⁹
19	C	By default, consumers CANNOT reshare data. The provider must explicitly enable this by setting ALLOW_RESHARING = TRUE on the share object.[¹⁵ [16]
20	B	A Data Clean Room is a privacy-preserving sharing framework where the provider defines allowed query patterns — enabling collaborative analysis without exposing raw sensitive data to the other party.[¹⁵ [19]

References

1. [Free Practice Questions for Snowflake COF-C03 Certification](#) - Study with 563 exam-style practice questions designed to help you prepare for the Snowflake SnowPro ...
2. [SnowPro® Core Certification \(COF-C03\) - Snowflake University](#) - Access the Exam Study Guide. This guide provides information that will help you prepare for your cer...

3. [SnowPro Core Exam Prep 2026 - App Store - Apple](#) - Pass the Snowflake SnowPro Core exam with confidence. 2,100+ questions, progressive difficulty, and ...
4. [Understanding & using Time Travel | Snowflake Documentation](#) - If the Time Travel retention period is set to 0, any modified or deleted data is moved into Fail-safe...
5. [Storage costs for Time Travel and Fail-safe](#)
6. [Working with Temporary and Transient Tables](#) - Transient and temporary tables have no Fail-safe period. As a result, no additional data storage cha...
7. [Snowflake time travel 101: A comprehensive overview \(2026\) - Flexera](#) - The maximum Time Travel retention for transient and ...
8. [Snowflake - Zero-Copy Cloning - Working Session](#) - You can get all snowflake Videos, PPTs, Queries, Interview questions and Practice files in my Udemy ...
9. [Migrating Time Travel history when Cloning an object with Time Travel](#) - Is there an option 3), which allows the time travel to remain with the original table rather than be...
10. [Cloning considerations | Snowflake Documentation](#) - For Iceberg tables that use Snowflake-provided storage (`EXTERNAL_VOLUME = 'SNOWFLAKE_MANAGED'`), CR...
11. [Introduction to business continuity & disaster recovery](#) - This topic describes the main use cases for replication and failover across regions and cloud platfo...
12. [Cross Cloud Business Continuity With Snowflake](#) - Implement cross-cloud business continuity with Snowflake replication for disaster recovery, multi-re...
13. [Introduction to replication and failover across multiple accounts](#)
14. [Apr 7, 2026: Workspaces replication \(General availability\)](#) - Files can be executed but not modified. When a secondary failover group is promoted to primary, all ...
15. [About Secure Data Sharing - Snowflake Documentation](#) - Secure Data Sharing lets you share selected objects in a database in your account with other Snowfla...
16. [Create and configure shares](#)
17. [Create and configure shares | Snowflake Documentation](#) - This topic describes the tasks associated with a data provider account creating and configuring shar...
18. [About listings | Snowflake Documentation](#) - A listing is an enhanced method of Secure Data Sharing and uses the same provider and consumer model...
19. [Data sharing and collaboration in Snowflake](#) - If you don't want to share data using a listing, you can use a direct share instead, see Secure data...
20. [8 - Cloning, Time travel, Replication & Secure Data Sharing features](#) - Provider account can create a maximum of 20 reader accounts only. Credits - Snowflakes official docu...